

Université de Toulon

SeaTech

Maîtriser les méthodes numériques de résolution d'équations différentielles

TP de calcul scientifique

Auteur : FLINOIS Mathias, GOUDON Lysandre,
ELHABTI Imad, RODRIGUEZ Duncan

Formation : 1ère année FISE 2025/26

Encadrant / Enseignant : ERSOY Mehmet

Date : May 31, 2026

Abstract

This report presents a study of numerical methods for solving ordinary differential equations, conducted as part of the practical work in numerical analysis. The aim is to compare the performance of the explicit and implicit Euler methods, Heun's method, the second-order Runge-Kutta method, and the fourth-order Runge-Kutta method. The fundamental concepts of consistency, stability, and convergence are theoretically examined and then numerically verified using MATLAB simulations. The obtained results illustrate the divergence phenomena of explicit schemes for certain time steps, as well as the superior accuracy of the Runge-Kutta methods.

Keywords: numerical analysis, ordinary differential equations, Euler method, Heun method, Runge-Kutta methods, stability, convergence, consistency.

Résumé

Ce rapport présente une étude des méthodes numériques de résolution des équations différentielles ordinaires réalisée dans le cadre des travaux pratiques d'analyse numérique. L'objectif est de comparer les performances des méthodes d'Euler explicite et implicite, de Heun, de Runge-Kutta d'ordre 2 et de Runge-Kutta d'ordre 4. Les notions fondamentales de consistance, stabilité et convergence sont étudiées théoriquement puis vérifiées numériquement à l'aide de simulations MATLAB. Les résultats obtenus permettent d'illustrer les phénomènes de divergence des schémas explicites pour certains pas de temps ainsi que la précision supérieure des méthodes de Runge-Kutta.

Mots-clés : analyse numérique, équations différentielles ordinaires, méthode d'Euler, méthode de Heun, méthodes de Runge-Kutta, stabilité, convergence, consistance.

Table des matières

1	Introduction	4
1.1	Contexte et état de l'art	4
1.2	Objectif du rapport	4
1.3	Organisation du rapport	4
I	Introduction à la stabilité et comparaison Euler Explicite/Implicite	5
II	Analyse de la convergence et précision des schémas à un pas	7
2	Méthode Euler Explicite	8
2.1	Consistance	8
2.2	Stabilité	8
2.3	Convergence	9
3	Méthode Heun Explicite	9
3.1	Consistance	9
3.1.1	Définition de l'erreur de consistance	9
3.1.2	Limite du taux d'accroissement de la solution exacte	9
3.1.3	Limite de l'incrément numérique Φ	9
3.1.4	Conclusion	10
3.2	Stabilité	10
3.3	Convergence	10
4	Méthode Runge-Kutta d'ordre 2 explicite	10
4.1	Définitions	10
4.2	Preuve de la consistance	10
4.3	Preuve de la stabilité	11
4.4	Preuve de la convergence	11
5	Méthode Runge-Kutta d'ordre 4 explicite	11

5.1 Définitions	11
5.2 Preuve de la consistance	12
5.3 Preuve de la stabilité	12
5.4 Preuve de la convergence	12
6 Estimation numérique de l'ordre de convergence des quatre méthodes	12
III Méthode implicite et explicite pour un second membre non linéaire	13
7 Méthode Euler Implicite	13
7.1 Stabilité	14
7.2 Convergence	15
8 Méthode Euler Explicite	15
8.1 Consistance	15
8.2 Stabilité	16
8.3 Convergence	16
9 Estimation numérique de l'ordre de convergence des deux méthodes	16
10 Conclusion	18
IV Conclusion	18
A Annexe : Programmes Matlab	19

1 Introduction

1.1 Contexte et état de l'art

Les équations différentielles ordinaires interviennent dans de nombreux domaines scientifiques et industriels tels que la mécanique, l'électronique, la physique ou encore la modélisation des systèmes dynamiques. Dans la majorité des cas, les solutions exactes de ces équations ne peuvent pas être déterminées analytiquement, ce qui rend nécessaire l'utilisation de méthodes numériques d'approximation.

Parmi les premières méthodes développées figure la méthode d'Euler, introduite au XVIII^e siècle, qui constitue la base des schémas numériques à un pas. Malgré sa simplicité de mise en œuvre, cette méthode présente des limitations importantes en termes de stabilité et de précision. Afin d'améliorer les approximations obtenues, plusieurs méthodes plus performantes ont été développées, notamment la méthode de Heun ainsi que les méthodes de Runge-Kutta d'ordre supérieur.

L'analyse moderne des schémas numériques repose principalement sur trois propriétés fondamentales : la consistance, qui mesure la qualité locale de l'approximation ; la stabilité, qui garantit le contrôle des erreurs numériques ; et la convergence, qui assure que la solution approchée tend vers la solution exacte lorsque le pas de discrétisation devient petit.

1.2 Objectif du rapport

L'objectif de ce rapport est d'étudier plusieurs méthodes numériques de résolution des équations différentielles ordinaires et d'analyser leurs propriétés mathématiques et numériques. Plus précisément, il s'agit :

- d'implémenter les méthodes d'Euler explicite et implicite, de Heun, de Runge-Kutta d'ordre 2 et de Runge-Kutta d'ordre 4 ;
- d'étudier théoriquement la consistance, la stabilité et la convergence de ces schémas numériques ;
- de comparer les solutions numériques obtenues avec les solutions exactes ;
- d'estimer numériquement les ordres de convergence des différentes méthodes à l'aide de simulations MATLAB ;
- d'illustrer les phénomènes de divergence et les limitations de stabilité de certains schémas explicites.

1.3 Organisation du rapport

Ce rapport est organisé en plusieurs parties.

La première partie introduit les notions de stabilité des schémas numériques à travers l'étude comparative des méthodes d'Euler explicite et implicite appliquées à une équation différentielle linéaire.

La deuxième partie est consacrée à l'analyse théorique et numérique de plusieurs schémas à un pas : Euler explicite, Heun, Runge-Kutta d'ordre 2 et Runge-Kutta d'ordre 4. Les propriétés de consistance, stabilité et convergence y sont démontrées puis validées expérimentalement par l'étude des ordres de convergence.

La troisième partie traite l'application des méthodes d'Euler explicite et implicite à une équation différentielle non linéaire. Les résultats numériques obtenus sont analysés afin d'évaluer l'influence de la non-linéarité sur la stabilité et la convergence des méthodes.

Enfin, une conclusion générale synthétise les principaux résultats obtenus ainsi que les limites des méthodes étudiées.

Première partie

Introduction à la stabilité et comparaison Euler Explicite/Implicite

$t_0, T \in \mathbb{R}, \quad t_0 < T, \quad$ et soit $f : [t_0, T] \times \mathbb{R} \rightarrow \mathbb{R}$ une fonction continue et Lipschitz par rapport à x .

On cherche à calculer une solution approchée $x_n \approx x(t_n)$ de l'équation

$$x'(t) = f(t, x(t)).$$

tel que

$$x'(t) = -2x(t), \quad x(0) = 1, \quad t \in [0, 10].$$

$$\frac{x'(t)}{x(t)} = -2$$

$$\ln |x(t)| = -2t + C$$

$$x(t) = Ce^{-2t}$$

et donc grace à la condition initiale $x(0)=1$

$$x(t) = e^{-2t}.$$

On souhaite appliquer la méthode d'Euler explicite et implicite à cette équation différentielle :

On rappelle dans un premier temps que $x_{n+1} = x_n + dt * f(t, x_n)$ et en traçant à l'aide d'un programme MATLAB les solutions de cette équation différentielle avec les différentes méthodes :

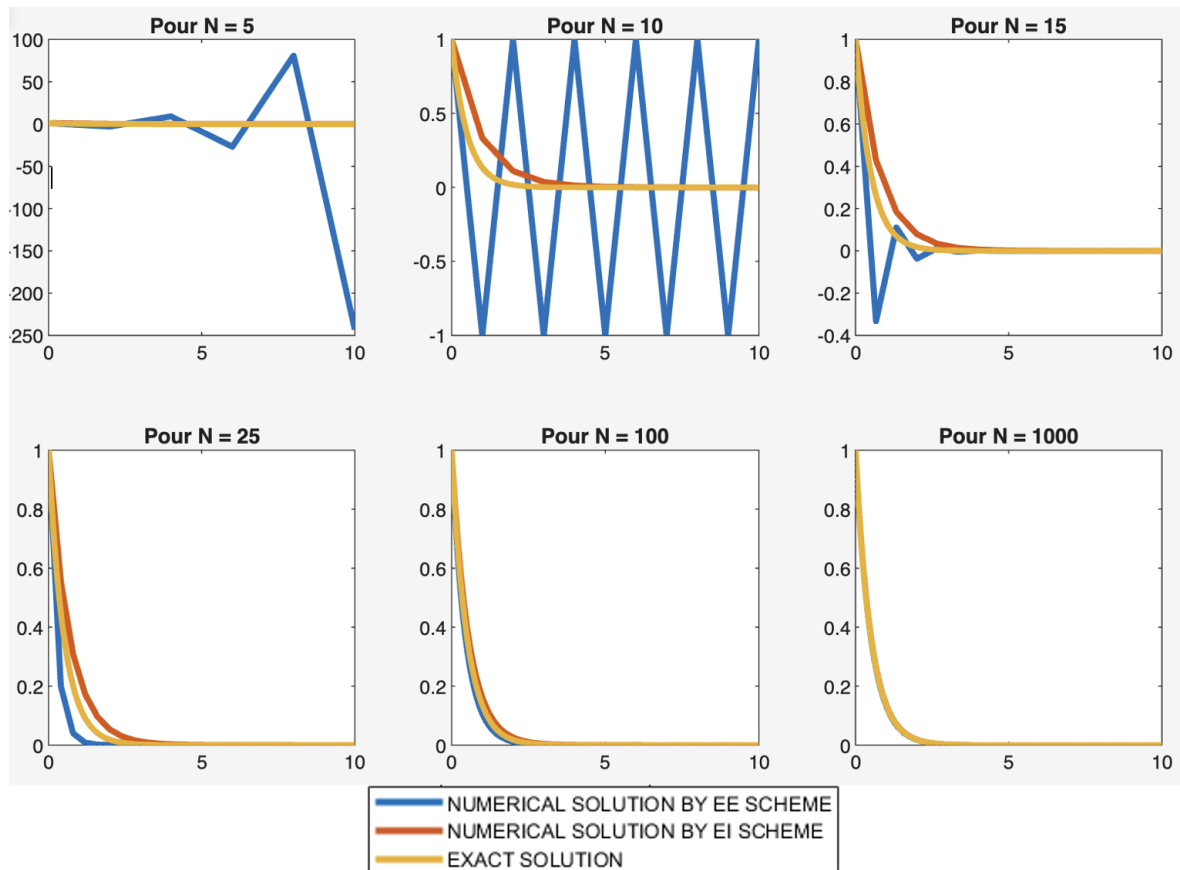
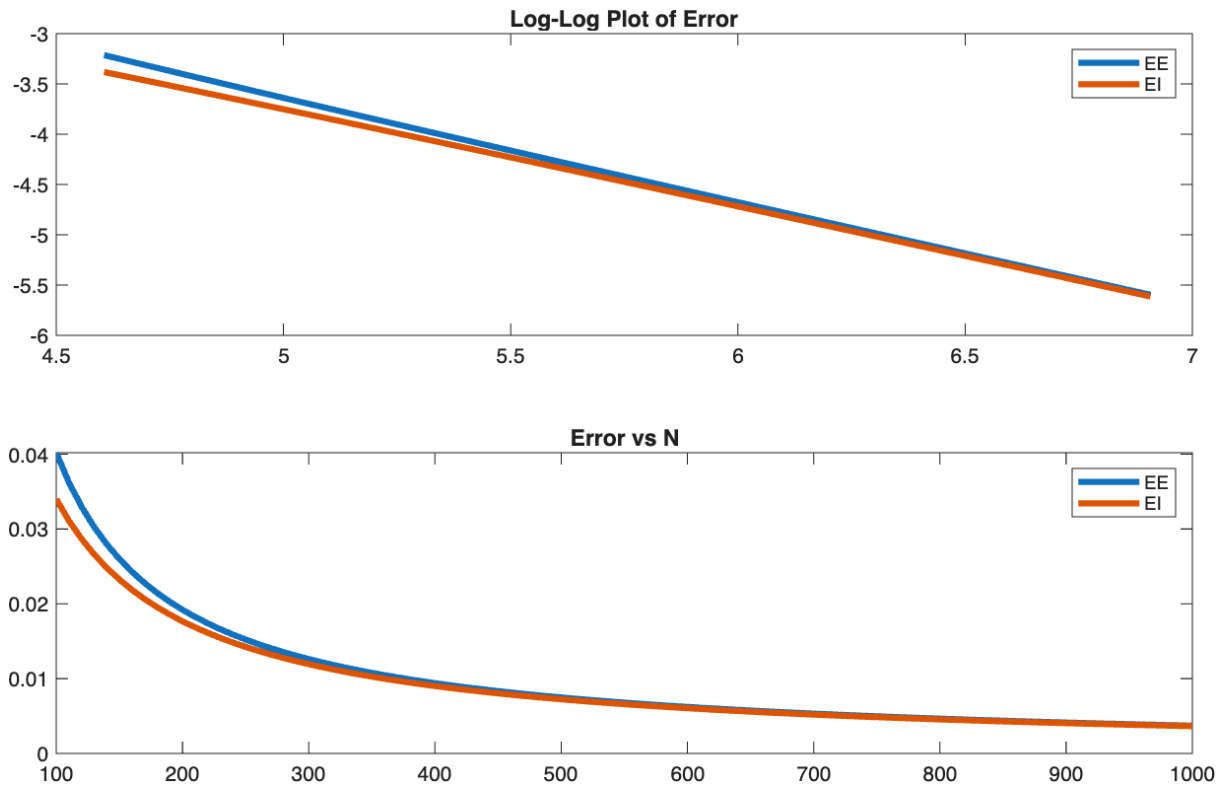


FIGURE 1 : Graphe de comparaison entre la solution exacte et la méthode d'Euler explicite et implicite pour des nombres de pas de temps différents

Alors pour $N = 5$, les deux méthodes divergent devant la solution exacte et on voit, quand on augmente le nombre de pas de temps, les deux méthodes convergent de plus en plus vite vers la solution exacte. Cependant, la méthode implicite converge beaucoup plus vite que la méthode explicite et donc a une meilleure stabilité numérique.

FIGURE 2 : Graphe de l'analyse de l'erreur en fonction de N

De plus, les deux courbes représentent l'évolution des erreurs EE et EI lorsque le nombre de points N augmente. On observe que les erreurs diminuent quand N augmente, ce qui indique une convergence de la méthode numérique.

Sur le graphique en échelle logarithmique (en haut), les courbes sont quasiment des droites parallèles. Une pente proche de -1 indique que l'erreur décroît approximativement comme $1/N$, ce qui est caractéristique d'un schéma numérique d'ordre 1.

En conclusion, la méthode d'Euler implicite présente une meilleure stabilité numérique que la méthode d'Euler explicite. Pour une même précision, elle permet généralement d'utiliser des pas de temps plus grands tout en conservant une erreur faible par rapport à la solution exacte. Elle constitue donc une méthode plus robuste pour la résolution numérique de certaines équations différentielles.

Deuxième partie

Analyse de la convergence et précision des schémas à un pas

2 Méthode Euler Explicite

2.1 Consistance

L'erreur de troncature e_n pour la méthode d'Euler explicite est définie par :

$$e_n = \frac{x(t_{n+1}) - x(t_n)}{\delta t} - f(t_n, x(t_n))$$

En effectuant un développement de Taylor de la solution exacte $x(t_{n+1}) = x(t_n + \delta t)$ au voisinage de t_n , on obtient :

$$x(t_{n+1}) = x(t_n) + \delta t \cdot x'(t_n) + \frac{\delta t^2}{2} x''(t_n) + \mathcal{O}(\delta t^3)$$

En réarrangeant les termes pour faire apparaître le taux d'accroissement :

$$\frac{x(t_{n+1}) - x(t_n)}{\delta t} = x'(t_n) + \frac{\delta t}{2} x''(t_n) + \mathcal{O}(\delta t^2)$$

En substituant cette expression dans la formule de l'erreur de troncature e_n :

$$e_n = \left(x'(t_n) + \frac{\delta t}{2} x''(t_n) + \mathcal{O}(\delta t^2) \right) - f(t_n, x(t_n))$$

Comme $x(t)$ est la solution exacte, elle vérifie $x'(t_n) = f(t_n, x(t_n))$. Les termes du premier ordre s'annulent donc, ce qui donne :

$$e_n = \frac{\delta t}{2} x''(t_n) + \mathcal{O}(\delta t^2)$$

Ainsi, l'erreur $e_n = \mathcal{O}(\delta t)$, donc la méthode est **consistante** d'ordre 1.

2.2 Stabilité

La méthode d'Euler explicite s'écrit sous la forme générale d'un schéma à un pas :

$$x_{n+1} = x_n + \delta t \cdot \Phi(t_n, x_n, \delta t)$$

où la fonction d'incrément est $\Phi(t_n, x_n, \delta t) = f(t_n, x_n)$.

Ce schéma est stable si sa fonction d'incrément est lipschitzienne en x . On montre d'abord que f est localement lipschitzienne en x .

Pour tout $(x, y) \in [-M, M]^2$, $\forall t \in [0, 1]$, $M \in \mathbb{R}$:

$$\begin{aligned} |f(t, x) - f(t, y)| &= |e^t - e^t| \\ &= 0 \end{aligned}$$

La méthode d'Euler explicite est donc **stable**.

2.3 Convergence

Ainsi, puisque nous avons la consistance et la stabilité de la méthode, nous pouvons en conclure que la méthode converge.

3 Méthode Heun Explicite

Le schéma de Heun est défini par le passage de x_n à x_{n+1} avec un pas h :

$$\begin{cases} \tilde{x}_{n+1} = x_n + \delta t f(t_n, x_n) \\ x_{n+1} = x_n + \frac{\delta t}{2} [f(t_n, x_n) + f(t_{n+1}, \tilde{x}_{n+1})] \end{cases}$$

3.1 Consistance

3.1.1 Définition de l'erreur de consistance

Pour un schéma à un pas de la forme $x_{n+1} = x_n + \delta t \Phi(t_n, x_n, \delta t)$, l'erreur de consistance ϵ_n est définie par :

$$\epsilon_n = \frac{x(t_{n+1}) - x(t_n)}{\delta t} - \Phi(t_n, x_n, \delta t) \quad (1)$$

Le schéma est dit **consistant** si $\lim_{\delta t \rightarrow 0} \epsilon_n = 0$.

3.1.2 Limite du taux d'accroissement de la solution exacte

Par définition de la dérivée de la solution exacte au point t , et sachant que $x'(t) = e^t$, on a directement :

$$\lim_{\delta t \rightarrow 0} \frac{x(t + \delta t) - x(t)}{\delta t} = x'(t) = e^t \quad (2)$$

3.1.3 Limite de l'incrément numérique Φ

Pour la méthode de Heun avec $f(t, x) = e^t$, la fonction d'incrément est définie par :

$$\Phi(t, x, \delta t) = \frac{1}{2} [e^t + e^{t+\delta t}] \quad (3)$$

Par continuité de la fonction exponentielle, sa limite lorsque $\delta t \rightarrow 0$ est :

$$\lim_{\delta t \rightarrow 0} \Phi(t, x, \delta t) = \frac{1}{2} [e^t + e^t] = e^t \quad (4)$$

3.1.4 Conclusion

En passant à la limite dans l'expression de l'erreur de consistance (1), on obtient :

$$\lim_{\delta t \rightarrow 0} \epsilon_n = e^t - e^t = 0 \quad (5)$$

La condition $\lim_{\delta t \rightarrow 0} \epsilon_n = 0$ étant vérifiée, la méthode de Heun est **consistante**.

3.2 Stabilité

La démonstration de la stabilité est similaire à celle de la méthode d'Euler explicite. On montre que la fonction $f(t, x) = e^t$ est lipschitzienne par rapport à x .

La méthode de Heun explicite est donc **stable**.

3.3 Convergence

Ainsi, puisque nous avons la consistance et la stabilité de la méthode, nous pouvons en conclure que la méthode converge.

4 Méthode Runge-Kutta d'ordre 2 explicite

4.1 Définitions

La méthode de Runge-Kutta d'ordre 2 (RK2, méthode du point milieu explicite) s'écrit avec un pas $\delta t > 0$ sous la forme :

$$x_{n+1} = x_n + \delta t \cdot \Phi(t_n, x_n, h) \quad (6)$$

où la fonction d'incrément Φ est définie par :

$$\Phi(t, x, \delta t) = f\left(t + \frac{\delta t}{2}, x + \frac{\delta t}{2} f(t, x)\right) \quad (7)$$

4.2 Preuve de la consistance

Une méthode numérique à un pas est dite **consistante** si sa fonction d'incrément vérifie :

$$\lim_{\delta t \rightarrow 0} \Phi(t, x, \delta t) = f(t, x) \quad (8)$$

La fonction f étant continue, on peut calculer la limite lorsque $\delta t \rightarrow 0$:

$$\begin{aligned}\lim_{\delta t \rightarrow 0} \Phi(t, x, h) &= \lim_{\delta t \rightarrow 0} f\left(t + \frac{h}{2}, x + \frac{h}{2}f(t, x)\right) \\ &= f\left(\lim_{\delta t \rightarrow 0} \left(t + \frac{h}{2}\right), \lim_{\delta t \rightarrow 0} \left(x + \frac{h}{2}f(t, x)\right)\right) \\ &= f(t, x)\end{aligned}$$

Puisque $\Phi(t, x, 0) = f(t, x)$, la condition de consistance est vérifiée.

4.3 Preuve de la stabilité

La démonstration de la stabilité est similaire à celle de la méthode d'Euler explicite. On montre que la fonction $f(t, x) = e^t$ est lipschitzienne par rapport à x .

La méthode Runge-Kutta d'ordre 2 explicite est donc **stable**.

4.4 Preuve de la convergence

Ainsi, puisque nous avons la consistance et la stabilité de la méthode, nous pouvons en conclure que la méthode Runge-Kutta d'ordre 2 converge.

5 Méthode Runge-Kutta d'ordre 4 explicite

5.1 Définitions

La méthode de Runge-Kutta d'ordre 4 classique (RK4 explicite) s'écrit avec un pas $\delta t > 0$ sous la forme :

$$x_{n+1} = x_n + \delta t \cdot \Phi(t_n, x_n, \delta t) \quad (9)$$

où la fonction d'incrément Φ est une moyenne pondérée de quatre pentes intermédiaires :

$$\Phi(t, x, \delta t) = \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (10)$$

avec les coefficients définis par :

$$k_1 = f(t, x) \quad (11)$$

$$k_2 = f\left(t + \frac{\delta t}{2}, x + \frac{\delta t}{2}k_1\right) \quad (12)$$

$$k_3 = f\left(t + \frac{\delta t}{2}, x + \frac{\delta t}{2}k_2\right) \quad (13)$$

$$k_4 = f(t + \delta t, x + \delta tk_3) \quad (14)$$

5.2 Preuve de la consistance

Une méthode numérique à un pas est dite **consistante** si sa fonction d'incrément vérifie :

$$\lim_{\delta t \rightarrow 0} \Phi(t, x, \delta t) = f(t, x) \quad (15)$$

La fonction f étant continue, calculons la limite de chaque coefficient lorsque $\delta t \rightarrow 0$:

$$\begin{aligned} \lim_{\delta t \rightarrow 0} k_1 &= f(t, x) \\ \lim_{\delta t \rightarrow 0} k_2 &= f(t + 0, x + 0) = f(t, x) \\ \lim_{\delta t \rightarrow 0} k_3 &= f(t + 0, x + 0) = f(t, x) \\ \lim_{\delta t \rightarrow 0} k_4 &= f(t + 0, x + 0) = f(t, x) \end{aligned}$$

En injectant ces limites dans l'expression de la fonction d'incrément Φ , on obtient :

$$\begin{aligned} \lim_{\delta t \rightarrow 0} \Phi(t, x, \delta t) &= \frac{1}{6} [f(t, x) + 2f(t, x) + 2f(t, x) + f(t, x)] \\ &= \frac{1}{6} \cdot 6f(t, x) = f(t, x) \end{aligned}$$

Puisque $\lim_{\delta t \rightarrow 0} \Phi(t, x, \delta t) = f(t, x)$, la condition de consistance est vérifiée.

5.3 Preuve de la stabilité

La démonstration de la stabilité est similaire à celle de la méthode d'Euler explicite. On montre que la fonction $f(t, x) = e^t$ est lipschitzienne par rapport à x .

La méthode de Runge-Kutta d'ordre 4 explicite est donc **stable**.

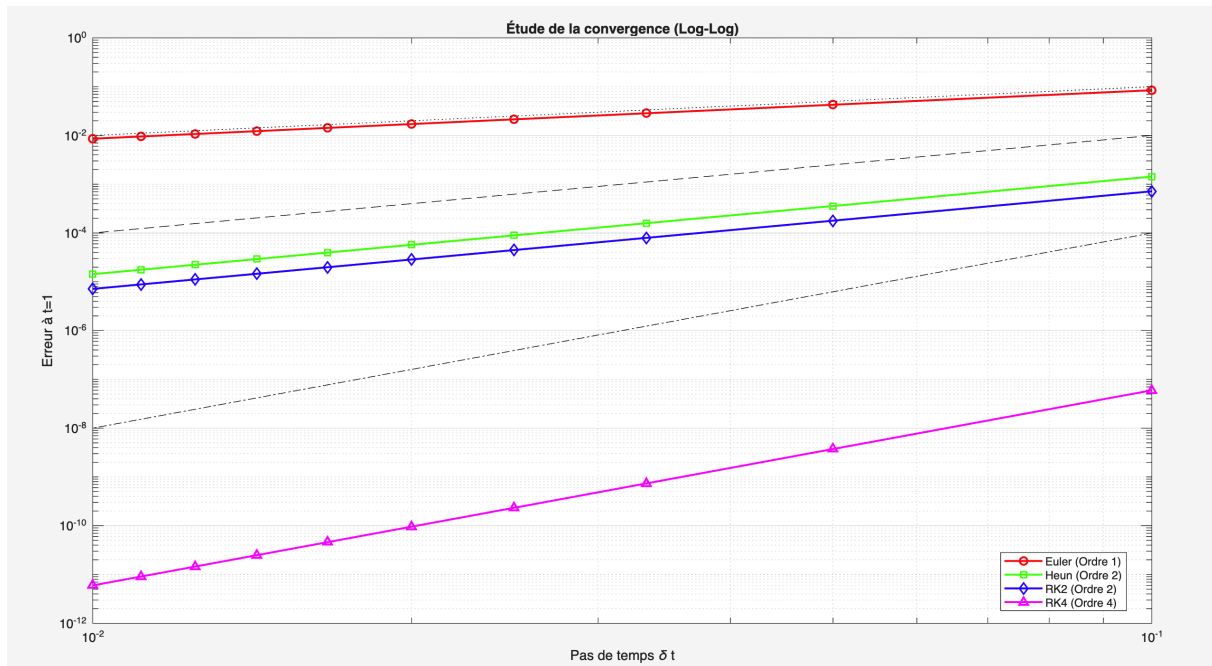
5.4 Preuve de la convergence

Ainsi, puisque nous avons démontré la consistance et la stabilité, nous pouvons en conclure que la méthode Runge-Kutta d'ordre 4 explicite **converge**.

6 Estimation numérique de l'ordre de convergence des quatre méthodes

On trace numériquement le graphe log-log de l'erreur E sur MATLAB en fonction de δt , le pas de la subdivision régulière.

Nous avons obtenus ces graphiques pour les quatre méthodes avec un pas de temps $N = 100$:

FIGURE 3 : Graphe log-log de l'erreur en fonction de δt TABLE 1 : Comparaison des ordres de convergence théoriques et numériques pour $f(t, x) = e^t$

Méthode Numérique	Ordre Théorique	Ordre Numérique Obtenu
Euler Explicite	1	0,99
Heun	2	2,00
Runge-Kutta 2 (RK2)	2	2,00
Runge-Kutta 4 (RK4)	4	4,00

Troisième partie

Méthode implicite et explicite pour un second membre non linéaire

7 Méthode Euler Implicite

Le problème de Cauchy posé sur $[0, 1]$ est :

$$\begin{cases} x'(t) = f(t, x(t)) = e^t x(t)^2 \\ x(0) = 0.5 \end{cases}$$

Le méthode d'Euler implicite, avec un pas de temps $\delta t = \frac{T}{N}$, $T \in \mathbb{R}$, $N \in \mathbb{N}$, est défini par :

$$x_{n+1} = x_n + \delta t \cdot f(t_{n+1}, x_{n+1})$$

Consistance

L'erreur de troncature locale e_n pour la méthode d'Euler implicite est définie en injectant la solution exacte $x(t)$ dans le schéma numérique :

$$e_n = \frac{x(t_{n+1}) - x(t_n)}{\delta t} - f(t_{n+1}, x(t_{n+1}))$$

En effectuant un développement de Taylor de la solution exacte $x(t_n) = x(t_{n+1} - \delta t)$ au voisinage de t_{n+1} , on obtient :

$$x(t_n) = x(t_{n+1}) - \delta t \cdot x'(t_{n+1}) + \frac{\delta t^2}{2} x''(t_{n+1}) + \mathcal{O}(\delta t^3)$$

En réarrangeant les termes pour faire apparaître le taux d'accroissement :

$$\frac{x(t_{n+1}) - x(t_n)}{\delta t} = x'(t_{n+1}) - \frac{\delta t}{2} x''(t_{n+1}) + \mathcal{O}(\delta t^2)$$

En substituant cette expression dans la formule de l'erreur de troncature e_n :

$$e_n = \left(x'(t_{n+1}) - \frac{\delta t}{2} x''(t_{n+1}) + \mathcal{O}(\delta t^2) \right) - f(t_{n+1}, x(t_{n+1}))$$

Comme $x(t)$ est la solution exacte de l'équation différentielle, elle vérifie $x'(t_{n+1}) = f(t_{n+1}, x(t_{n+1}))$. Les termes du premier ordre s'annulent donc, ce qui donne :

$$e_n = -\frac{\delta t}{2} x''(t_{n+1}) + \mathcal{O}(\delta t^2)$$

Ainsi, l'erreur $e_n = \mathcal{O}(\delta t^2)$, donc la méthode est **consistante** à l'ordre 1.

7.1 Stabilité

La méthode d'Euler implicite s'écrit sous la forme générale d'un schéma à un pas :

$$x_{n+1} = x_n + \delta t \cdot \Phi(t_n, x_n, \delta t)$$

où la fonction d'incrément est $\Phi(t_n, x_n, \delta t) = f(t_{n+1}, x_{n+1})$.

Ce schéma est stable si sa fonction d'incrément est lipschitzienne en x . On montre d'abord que f est localement lipschitzienne en x .

Pour tout $(x, y) \in [-M, M]^2$, $\forall t \in [0, 1]$, $M \in \mathbb{R}$:

$$\begin{aligned} |f(t, x) - f(t, y)| &= |e^t x^2 - e^t y^2| \\ &= e^t |x^2 - y^2| \\ &= e^t |x + y| |x - y|. \end{aligned}$$

Or, comme $x, y \in [-M, M]$, on a $|x + y| \leq 2M$, et puisque $t \in [0, 1]$, $e^t \leq e$.

Ainsi :

$$|f(t, x) - f(t, y)| \leq 2eM |x - y|$$

f est donc localement lipschitzienne en x , uniformément en $t \in [0, 1]$, avec constante $2eM$.

La fonction d'incrément $\Phi(t_n, x_n, \delta t) = f(t_{n+1}, x_{n+1})$ est ainsi aussi lipschitzienne en x_n avec la même constante $2eM$. La méthode d'Euler implicite est donc **stable**.

7.2 Convergence

Ainsi, puisque nous avons la consistance et la stabilité de la méthode, nous pouvons en conclure que la méthode **converge**.

8 Méthode Euler Explicite

Le schéma d'Euler explicite, avec un pas de temps $\delta t = \frac{T}{N}$, est défini par :

$$x_{n+1} = x_n + \delta t \cdot f(t_n, x_n)$$

8.1 Consistance

L'erreur de troncature e_n pour la méthode d'Euler explicite est définie par :

$$e_n = \frac{x(t_{n+1}) - x(t_n)}{\delta t} - f(t_n, x(t_n))$$

En effectuant un développement de Taylor de la solution exacte $x(t_{n+1}) = x(t_n + \delta t)$ au voisinage de t_n , on obtient :

$$x(t_{n+1}) = x(t_n) + \delta t \cdot x'(t_n) + \frac{\delta t^2}{2} x''(t_n) + \mathcal{O}(\delta t^3)$$

En réarrangeant les termes pour faire apparaître le taux d'accroissement :

$$\frac{x(t_{n+1}) - x(t_n)}{\delta t} = x'(t_n) + \frac{\delta t}{2} x''(t_n) + \mathcal{O}(\delta t^2)$$

En substituant cette expression dans la formule de l'erreur de troncature e_n :

$$e_n = \left(x'(t_n) + \frac{\delta t}{2} x''(t_n) + \mathcal{O}(\delta t^2) \right) - f(t_n, x(t_n))$$

Comme $x(t)$ est la solution exacte, elle vérifie $x'(t_n) = f(t_n, x(t_n))$. Les termes du premier ordre s'annulent donc, ce qui donne :

$$e_n = \frac{\delta t}{2} x''(t_n) + \mathcal{O}(\delta t^2)$$

Ainsi, l'erreur $e_n = \mathcal{O}(\delta t)$, donc la méthode est **consistante** d'ordre 1.

8.2 Stabilité

La méthode d'Euler explicite s'écrit sous la forme générale d'un schéma à un pas :

$$x_{n+1} = x_n + \delta t \cdot \Phi(t_n, x_n, \delta t)$$

où la fonction d'incrément est ici $\Phi(t, x, \delta t) = f(t, x)$.

La démonstration de la stabilité pour la méthode de Euler explicite est identique à celle de Euler implicite. La seule différence réside sur la fonction d'incrément :

$$\begin{cases} \Phi(t_n, x_n, \delta t) = f(t_n, x_n) & \text{(Euler explicite)} \\ \Phi(t_n, x_n, \delta t) = f(t_{n+1}, x_{n+1}) & \text{(Euler implicite)} \end{cases}$$

Ainsi, la fonction d'incrément est **localement lipschitzienne** en x . La méthode d'Euler explicite est ainsi **stable**.

8.3 Convergence

Ainsi, puisque nous avons la consistance et la stabilité de la méthode, nous pouvons en conclure que la méthode converge.

9 Estimation numérique de l'ordre de convergence des deux méthodes

Ci-dessous, nous avons tracé la reconstruction de la solution numérique avec les méthodes d'Euler implicite et explicite.

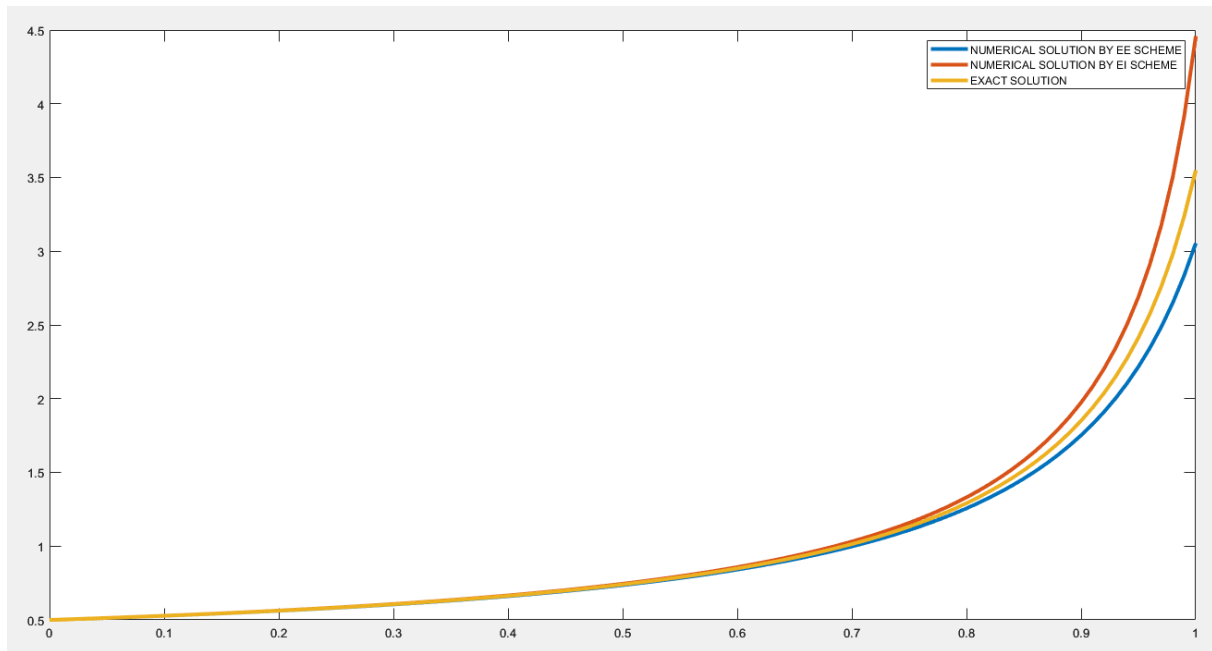
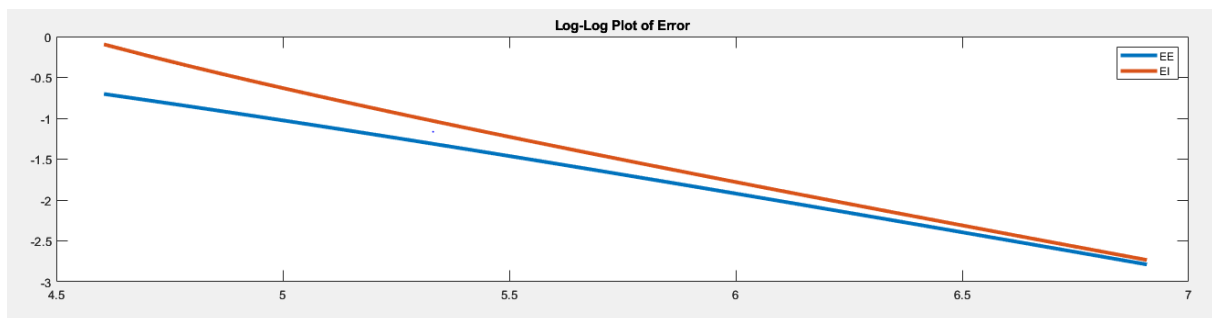


FIGURE 4 : Graphe de comparaisons des schémas numériques

Nous avons une reconstruction pour les deux méthodes qui semble correcte.

On trace maintenant le graphe log-log de l'erreur E sur MATLAB en fonction de N , le nombre de pas de temps.

Nous avons obtenus ces graphiques pour les deux méthodes avec un pas de temps $N = 100$:

FIGURE 5 : Graphe log-log de l'erreur en fonction de N

On lit deux points $(\log(N_1), \log(E(N_1)))$ et $(\log(N_2), \log(E(N_2)))$ sur la courbe et on estime la pente :

$$p \approx \left| \frac{\log(E(N_1)) - \log(E(N_2))}{\log(N_1) - \log(N_2)} \right|$$

Euler implicite

On prend les points $A_1(5.858, -1.626)$ et $B_1(6.745, -2.566)$, ce qui donne :

$$p \approx \left| \frac{-1.626 - (-2.566)}{5.858 - 6.745} \right| = \left| \frac{0.940}{-0.887} \right| \approx 1.059$$

On confirme ainsi numériquement l'ordre 1 de la méthode d'Euler implicite.

Euler explicite

On prend les points $A_2(5.829, -1.762)$ et $B_2(6.565, -2.457)$, ce qui donne :

$$p \approx \left| \frac{-1.762 - (-2.457)}{5.829 - 6.565} \right| = \left| \frac{0.695}{-0.736} \right| \approx 0.945$$

On confirme ainsi numériquement l'ordre 1 de la méthode d'Euler explicite.

10 Conclusion

Malgré un second membre non linéaire, nous avons retrouvés les ordres des méthodes Euler explicite et implicite.

Quatrième partie

Conclusion

Au cours de ce travail pratique, plusieurs méthodes numériques de résolution des équations différentielles ordinaires ont été étudiées et comparées. Les méthodes d'Euler explicite et implicite, de Heun ainsi que les méthodes de Runge-Kutta d'ordre 2 et 4 ont été analysées aussi bien d'un point de vue théorique que numérique.

L'étude a permis de mettre en évidence l'importance des propriétés de consistance, stabilité et convergence dans l'évaluation des performances d'un schéma numérique. Les simulations réalisées sous MATLAB ont confirmé les résultats théoriques concernant les ordres de convergence des différentes méthodes. En particulier, les méthodes de Runge-Kutta ont montré une précision nettement supérieure à celle de la méthode d'Euler explicite.

Les expériences numériques ont également illustré les limites de stabilité des méthodes explicites lorsque le pas de temps devient trop grand, tandis que les méthodes implicites se sont révélées plus robustes face aux problèmes de stabilité.

Enfin, ce travail a permis de mieux comprendre les enjeux liés au choix d'une méthode numérique adaptée selon la nature du problème étudié, le compromis recherché entre coût de calcul et précision, ainsi que l'influence du pas de discrétisation sur la qualité des résultats obtenus.

A Annexe : Programmes Matlab

Programme MATLAB de tracé du graphe de comparaison des schémas numériques et du graphe log-log de l'erreur.

```

1
2 clear all; clc; close all;
3
4 %
5 % =====
6 % Paramètres du problème (Exercice 2)
7 % =====
8 T = 1;           % Temps final [cite: 23]
9 x0 = 1;          % Condition initiale  $x(0) = 1$  [cite: 24]
10 N_init = 100;    % Nombre de pas pour le premier tracé
11 % Définition de  $f(t,x) = \exp(t)$  [cite: 24]
12 f = @(t,x) exp(t);
13
14 % Solution exacte :  $x(t) = \exp(t)$ 
15 % Car  $x'(t) = \exp(t)$  et  $x(0) = 1$  [cite: 24]
16 x_exacte = @(t) exp(t);
17
18 %
19 % =====
20 % 1. Visualisation des solutions numériques (N = 100)
21 % =====
22 [x_ee, t_ee] = ee(f, T, N_init, x0);
23 [x_he, t_he] = heun(f, T, N_init, x0);
24 [x_rk2, t_rk2] = rk2(f, T, N_init, x0);
25 [x_rk4, t_rk4] = rk4(f, T, N_init, x0);
26
27 t_fine = 0:0.01:T;
28 figure(1);
29 plot(t_fine, x_exacte(t_fine), 'k--', 'LineWidth', 2); hold on;
30 plot(t_ee, x_ee, 'r', t_he, x_he, 'g', t_rk2, x_rk2, 'b', t_rk4, x_rk4,
31      'm');
32 legend('Solution Exacte', 'Euler Explicite', 'Heun', 'RK2', 'RK4');
33 title('Comparaison des schémas numériques (N=100)');
34 xlabel('Temps t'); ylabel('x(t)');
35 grid on;

```

```

35
36 %
=====

37 % 2. tude de la convergence (Ordre numérique)
38 %
=====

39 NN = 10:10:100; % On fait varier N (donc dt) [cite: 30]
40 err_ee = zeros(size(NN));
41 err_he = zeros(size(NN));
42 err_rk2 = zeros(size(NN));
43 err_rk4 = zeros(size(NN));
44 dt_list = zeros(size(NN));
45
46 for i = 1:length(NN)
47     N = NN(i);
48     dt_list(i) = T/N;
49
50     % Calcul des solutions à t = T
51     [x1, ~] = ee(f, T, N, x0);
52     [x2, ~] = heun(f, T, N, x0);
53     [x3, ~] = rk2(f, T, N, x0);
54     [x4, ~] = rk4(f, T, N, x0);
55
56     % Calcul de l'erreur absolue à l'instant final t=1
57     sol_T = x_exacte(T);
58     err_ee(i) = abs(x1(end) - sol_T);
59     err_he(i) = abs(x2(end) - sol_T);
60     err_rk2(i) = abs(x3(end) - sol_T);
61     err_rk4(i) = abs(x4(end) - sol_T);
62 end
63
64 % Tracés log-log pour déterminer l'ordre de convergence
65 figure(2);
66 loglog(dt_list, err_ee, 'r-o', 'LineWidth', 1.5); hold on;
67 loglog(dt_list, err_he, 'g-s', 'LineWidth', 1.5);
68 loglog(dt_list, err_rk2, 'b-d', 'LineWidth', 1.5);
69 loglog(dt_list, err_rk4, 'm-^', 'LineWidth', 1.5);
70
71 % Ajout de droites de pente 1, 2 et 4 pour comparaison théorique
72 loglog(dt_list, dt_list, 'k:', 'HandleVisibility','off'); % Pente 1
73 loglog(dt_list, dt_list.^2, 'k--', 'HandleVisibility','off'); % Pente 2
74 loglog(dt_list, dt_list.^4, 'k-.', 'HandleVisibility','off'); % Pente 4
75
76 grid on;
77 xlabel('Pas de temps \delta t');
78 ylabel('Erreur à t=1');

```

```

79 title(' tude de la convergence (Log-Log)');
80 legend('Euler (Ordre 1)', 'Heun (Ordre 2)', 'RK2 (Ordre 2)', 'RK4 (Ordre
    4)', 'Location', 'best');
81
82 % Affichage des ordres calculés numériquement
83 fprintf('Ordre numérique Euler : %.2f\n', (log(err_ee(end))-log(err_ee
    (1)))/(log(dt_list(end))-log(dt_list(1))));
84 fprintf('Ordre numérique Heun : %.2f\n', (log(err_he(end))-log(err_he
    (1)))/(log(dt_list(end))-log(dt_list(1))));
85 fprintf('Ordre numérique RK2 : %.2f\n', (log(err_rk2(end))-log(err_rk2
    (1)))/(log(dt_list(end))-log(dt_list(1))));
86 fprintf('Ordre numérique RK4 : %.2f\n', (log(err_rk4(end))-log(err_rk4
    (1)))/(log(dt_list(end))-log(dt_list(1))));

```

Listing 1 : Programme MATLAB de tracé du graphe de comparaison des schémas numériques et du graphe log-log de l'erreur

Euler Explicite

```

1
2 function [x,t] = ee(f, T, N, x0)
3 % EE : Euler Explicite
4 % Entrées :
5 % f : fonction du type f(t, x) représentant l'EDO dx/dt = f(t, x)
6 % T : temps final
7 % N : nombre de pas de temps
8 % x0 : condition initiale x(0) = x0
9 % Sorties :
10 % x : vecteur des solutions approximatives aux temps t
11 % t : vecteur des temps
12
13 x(1) = x0; % Initialisation de la solution avec la condition
    initiale
14 dt = T / N; % Calcul du pas de temps
15
16 for n = 1:N
17     t(n) = (n - 1) * dt; % Calcul du temps courant
18     x(n + 1) = x(n) + dt * f(t(n), x(n)); % Formule d'Euler explicite :
    x_{n+1} = x_n + dt * f(t_n, x_n)
19 end
20
21 t(N + 1) = T; % Ajout du temps final dans le vecteur des temps
22
23 end

```

Listing 2 : Euler Explicite

Euler Implicite

1

```

2 function [x,t] = ei(f, df, T, N, x0)
3 % EI : Euler Implicite
4 % Entrées :
5 % f : fonction du type f(t, x) représentant l'EDO dx/dt = f(t, x)
6 % df : dérivée partielle de f par rapport à x, nécessaire pour Newton
7 % T : temps final
8 % N : nombre de pas de temps
9 % x0 : condition initiale x(0) = x0
10 % Sorties :
11 % x : vecteur des solutions approximatives aux temps t
12 % t : vecteur des temps
13
14 x(1) = x0; % Initialisation avec la condition initiale
15 dt = T / N; % Pas de temps
16
17 for n = 1:N
18     t(n) = (n - 1) * dt; % Temps courant
19
20     % Construction de l'équation implicite : G(X) = X - x(n) - dt * f(t(n) + dt, X)
21     G = @(X) (X - x(n) - dt * f(t(n) + dt, X)); % Fonction à annuler
22     dG = @(X) (1 - dt * df(t(n) + dt, X)); % Dérivée de G pour Newton
23
24     % == Méthode de Newton-Raphson ==
25     eps = 1e-6; % Tolérance pour le critère d'arrêt
26     kmax = 1e3; % Nombre maximal d'itérations
27     k = 0; % Compteur d'itérations
28     y = x(n); % Initialisation de Newton avec la valeur précédente
29
30     while (abs(G(y)) > eps) && (k < kmax)
31         y = y - G(y) / dG(y); % Itération de Newton
32         k = k + 1;
33     end
34
35     if k < kmax
36         x(n + 1) = y; % Solution trouvée
37     else
38         error('STOP') % Erreur : Newton n a pas convergé
39     end
40 end
41
42 t(N + 1) = T; % Ajout du temps final dans le vecteur des temps
43
44 end

```

Listing 3 : Euler Implicite

Heun Explicite

```
1
2 function [x,t] = heun(f, T, N, x0)
3     x(1) = x0;
4     dt = T / N;
5     t = 0:dt:T; % Créer le vecteur temps complet dès le départ est plus
6                 propre
7
8     for n = 1:N
9         % tape 1 : Prédicteur (Euler Explicite)
10        x_tilde = x(n) + dt * f(t(n), x(n));
11        % tape 2 : Correcteur (Moyenne des pentes)
12        x(n + 1) = x(n) + (dt/2) * (f(t(n), x(n)) + f(t(n+1), x_tilde));
13    end
end
```

Listing 4 : Heun Explicite

Runge-Kutta 2 Explicite

```
1
2 function [x,t] = rk2(f, T, N, x0)
3     x(1) = x0;
4     dt = T / N;
5     t = 0:dt:T;
6
7     for n = 1:N
8         k1 = f(t(n), x(n));
9         k2 = f(t(n) + dt/2, x(n) + (dt/2) * k1);
10        x(n + 1) = x(n) + dt * k2;
11    end
12 end
```

Listing 5 : Runge-Kutta 2 Explicite

Runge-Kutta 4 Explicite

```
1
2 function [x,t] = rk4(f, T, N, x0)
3     x(1) = x0;
4     dt = T / N;
5     t = 0:dt:T;
6
7     for n = 1:N
8         k1 = f(t(n), x(n));
9         k2 = f(t(n) + dt/2, x(n) + (dt/2) * k1);
10        k3 = f(t(n) + dt/2, x(n) + (dt/2) * k2);
11        k4 = f(t(n) + dt, x(n) + dt * k3);
12
13        x(n + 1) = x(n) + (dt/6) * (k1 + 2*k2 + 2*k3 + k4);
```

```
14     end
15 end
```

Listing 6 : Runge-Kutta 4 Explicite